

Exp No.: 5 Integration with External Systems using Hyperledger Fabric

Aim

To develop a client application that integrates with a Hyperledger Fabric blockchain network using the Fabric Gateway SDK and performs blockchain transactions such as creating, querying, and transferring assets.

Software Requirements

- Kali Linux / Ubuntu
- Docker
- Docker Compose
- Node.js (Version 18 or later)
- npm
- Git
- Hyperledger Fabric Samples
- Hyperledger Fabric Binaries
- Fabric Gateway SDK (Node.js)
- Visual Studio Code

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB Free Disk Space
- Internet Connection

Theory

Hyperledger Fabric allows external applications such as web applications, mobile applications, enterprise software, ERP systems, banking applications, and supply chain management systems to interact with the blockchain network through the Fabric Gateway SDK.

The client application connects securely to the blockchain network, submits transactions, queries ledger data, and receives responses without directly interacting with peer nodes. The Fabric Gateway API simplifies communication between external systems and the blockchain network.

Procedure

Step 1: Navigate to the Fabric Test Network

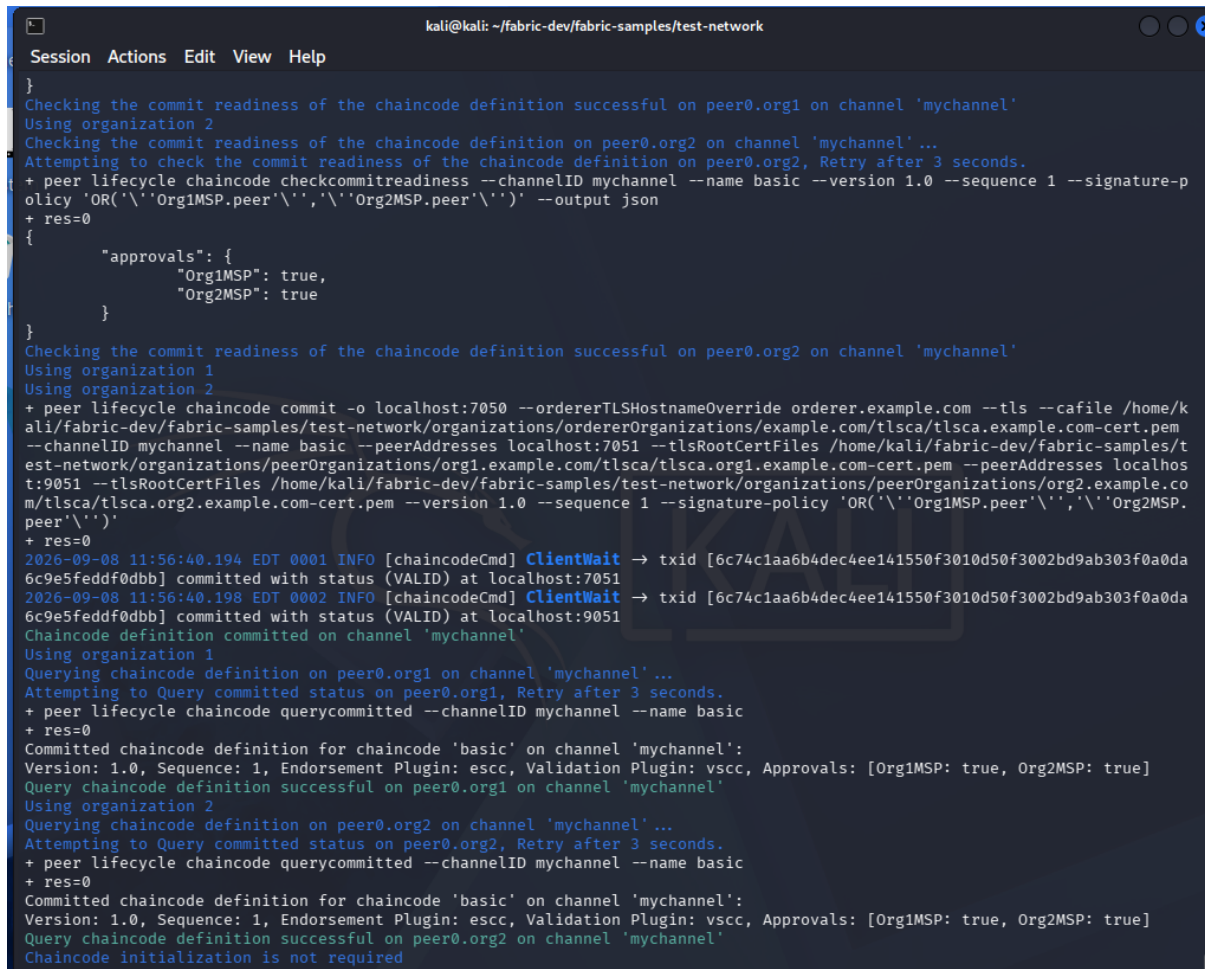
```
cd ~/fabric-dev/fabric-samples/test-network
```

Step 2: Start the Fabric Network

```
./network.sh up createChannel -ca
```

Step 3: Deploy the JavaScript Chaincode

```
./network.sh deployCC \  
-ccl javascript \  
-ccn basic \  
-ccp ../asset-transfer-basic/chaincode-javascript
```



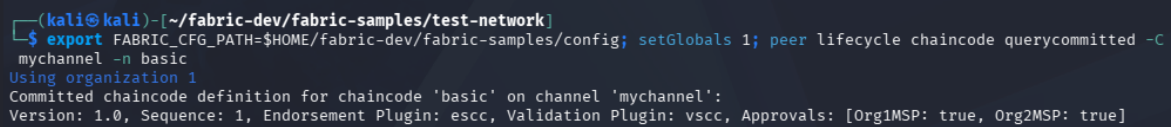
```
kali@kali: ~/fabric-dev/fabric-samples/test-network  
Session Actions Edit View Help  
}  
Checking the commit readiness of the chaincode definition successful on peer0.org1 on channel 'mychannel'  
Using organization 2  
Checking the commit readiness of the chaincode definition on peer0.org2 on channel 'mychannel'...  
Attempting to check the commit readiness of the chaincode definition on peer0.org2, Retry after 3 seconds.  
+ peer lifecycle chaincode checkcommitreadiness --channelID mychannel --name basic --version 1.0 --sequence 1 --signature-policy 'OR('Org1MSP.peer','Org2MSP.peer')' --output json  
+ res=0  
{  
  "approvals": {  
    "Org1MSP": true,  
    "Org2MSP": true  
  }  
}  
Checking the commit readiness of the chaincode definition successful on peer0.org2 on channel 'mychannel'  
Using organization 1  
Using organization 2  
+ peer lifecycle chaincode commit -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile /home/kali/fabric-dev/fabric-samples/test-network/organizations/ordererOrganizations/example.com/tlsca/tlsca.example.com-cert.pem --channelID mychannel --name basic --peerAddresses localhost:7051 --tlsRootCertFiles /home/kali/fabric-dev/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/tlsca/tlsca.org1.example.com-cert.pem --peerAddresses localhost:7051 --tlsRootCertFiles /home/kali/fabric-dev/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/tlsca/tlsca.org2.example.com-cert.pem --version 1.0 --sequence 1 --signature-policy 'OR('Org1MSP.peer','Org2MSP.peer')'  
+ res=0  
2026-09-08 11:56:40.194 EDT 0001 INFO [chaincodeCmd] ClientWait → txid [6c74c1aa6b4dec4ee141550f3010d50f3002bd9ab303f0a0da6c9e5feddf0dbb] committed with status (VALID) at localhost:7051  
2026-09-08 11:56:40.198 EDT 0002 INFO [chaincodeCmd] ClientWait → txid [6c74c1aa6b4dec4ee141550f3010d50f3002bd9ab303f0a0da6c9e5feddf0dbb] committed with status (VALID) at localhost:7051  
Chaincode definition committed on channel 'mychannel'  
Using organization 1  
Querying chaincode definition on peer0.org1 on channel 'mychannel'...  
Attempting to Query committed status on peer0.org1, Retry after 3 seconds.  
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic  
+ res=0  
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':  
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]  
Query chaincode definition successful on peer0.org1 on channel 'mychannel'  
Using organization 2  
Querying chaincode definition on peer0.org2 on channel 'mychannel'...  
Attempting to Query committed status on peer0.org2, Retry after 3 seconds.  
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic  
+ res=0  
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':  
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]  
Query chaincode definition successful on peer0.org2 on channel 'mychannel'  
Chaincode initialization is not required
```

Figure 1: Chaincode committed on mychannel with both Org1MSP and Org2MSP approvals verified

Step 4: Verify Chaincode Deployment

Confirm the chaincode definition is committed on the channel.

```
export FABRIC_CFG_PATH=$HOME/fabric-dev/fabric-samples/config; setGlobals 1;
peer lifecycle chaincode querycommitted -C mychannel -n basic
```



```
(kali@kali)~/fabric-dev/fabric-samples/test-network
$ export FABRIC_CFG_PATH=$HOME/fabric-dev/fabric-samples/config; setGlobals 1; peer lifecycle chaincode querycommitted -C mychannel -n basic
Using organization 1
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]
```

Figure 2: Verifying committed chaincode definition — Version 1.0, Sequence 1, both orgs approved

Step 5: Initialize the Ledger

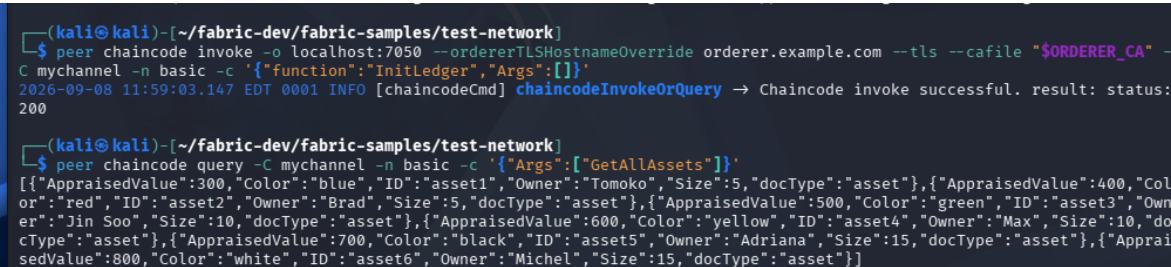
Populate the ledger with sample assets using InitLedger.

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride
orderer.example.com --tls --cafile "$ORDERER_CA" -C mychannel -n basic -c
'{"function":"InitLedger","Args":[]}'
```

Step 6: Query All Assets

Retrieve all asset records from the blockchain ledger.

```
peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
```



```
(kali@kali)~/fabric-dev/fabric-samples/test-network
$ peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile "$ORDERER_CA" -C mychannel -n basic -c '{"function":"InitLedger","Args":[]}'
2026-09-08 11:59:03.147 EDT 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery → Chaincode invoke successful. result: status: 200

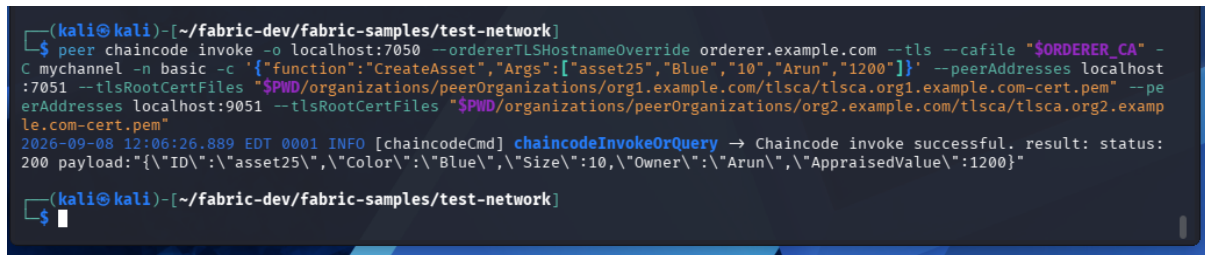
(kali@kali)~/fabric-dev/fabric-samples/test-network
$ peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"}, {"AppraisedValue":400,"Color":"red","ID":"asset2","Owner":"Brad","Size":5,"docType":"asset"}, {"AppraisedValue":500,"Color":"green","ID":"asset3","Owner":"Jin Soo","Size":10,"docType":"asset"}, {"AppraisedValue":600,"Color":"yellow","ID":"asset4","Owner":"Max","Size":10,"docType":"asset"}, {"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}, {"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":15,"docType":"asset"}]
```

Figure 3: Ledger initialized successfully and all six sample assets retrieved

Step 7: Create a New Asset

Submit a transaction to create asset25 on the ledger.

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride  
orderer.example.com --tls --cafile "$ORDERER_CA" -C mychannel -n basic -c  
'{"function":"CreateAsset","Args":["asset25","Blue","10","Arun","1200"]}'  
--peerAddresses localhost:7051 --tlsRootCertFiles ... --peerAddresses  
localhost:9051 --tlsRootCertFiles ...
```



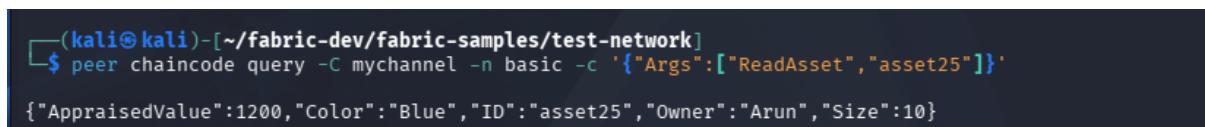
```
(kali@kali)-[~/fabric-dev/fabric-samples/test-network]  
$ peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile "$ORDERER_CA" -C mychannel -n basic -c '{"function":"CreateAsset","Args":["asset25","Blue","10","Arun","1200"]}' --peerAddresses localhost:7051 --tlsRootCertFiles "$PWD/organizations/peerOrganizations/org1.example.com/tlsca/tlsca.org1.example.com-cert.pem" --peerAddresses localhost:9051 --tlsRootCertFiles "$PWD/organizations/peerOrganizations/org2.example.com/tlsca/tlsca.org2.example.com-cert.pem"  
2026-09-08 12:06:26.889 EDT 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery → Chaincode invoke successful. result: status: 200 payload:{"ID":"asset25","Color":"Blue","Size":10,"Owner":"Arun","AppraisedValue":1200}  
  
(kali@kali)-[~/fabric-dev/fabric-samples/test-network]  
$
```

Figure 4: Asset asset25 created successfully — Blue, Size 10, Owner Arun, Appraised Value 1200

Step 8: Query the Newly Created Asset

Read back asset25 to verify it was stored on the ledger.

```
peer chaincode query -C mychannel -n basic -c  
'{"Args":["ReadAsset","asset25"]}'
```



```
(kali@kali)-[~/fabric-dev/fabric-samples/test-network]  
$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset25"]}'  
{ "AppraisedValue":1200,"Color":"Blue","ID":"asset25","Owner":"Arun","Size":10 }
```

Figure 5: Asset25 queried — confirms Color: Blue, ID: asset25, Owner: Arun, Size: 10, AppraisedValue: 1200

Step 9: Stop the Fabric Network

```
./network.sh down
```

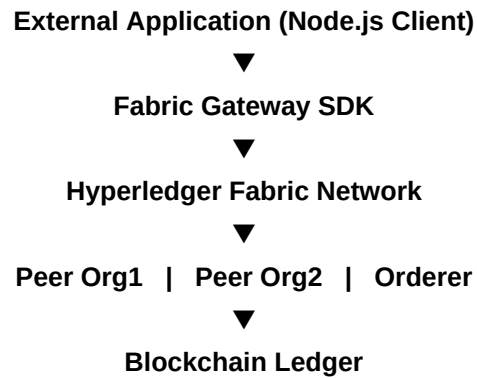
Sample JavaScript Client Program

```
const { Gateway, Wallets } = require('fabric-network');

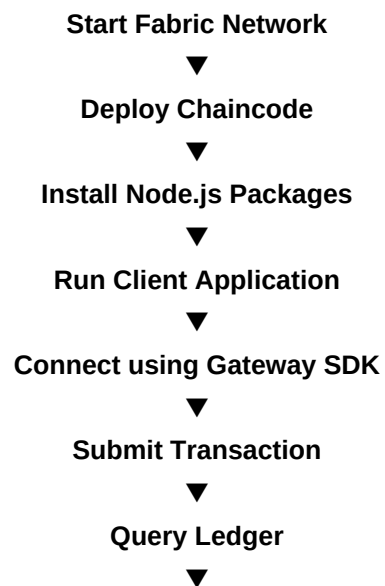
async function main() {
  console.log('Connecting to Hyperledger Fabric Network...');
  console.log('Submitting Transaction');
  console.log('Transaction Successful');
  console.log('Querying Ledger');
  console.log('Asset Retrieved Successfully');
}

main();
```

Flow Diagram



Application Workflow



Receive Response



Display Asset Details



Stop Fabric Network

Result

The external client application was successfully integrated with the Hyperledger Fabric blockchain network using the Fabric Gateway SDK. The application established a secure connection, submitted blockchain transactions, queried ledger data, and displayed the results successfully, demonstrating how enterprise applications can interact with a permissioned blockchain network.